TrustNet Secure | Cybersecurity Internship — CTF Assessment— HackerSchool

Executive Summary

This test targeted a practice/lab machine ("HackerSchool") at IP **192.168.1.10**. The tester scanned the machine, found a file server and web server open to the public, followed a trail of hidden clues left on the system, and ended up with full **root (administrator) access** — the highest level of control possible. Two separate weaknesses were exploited along the way: an anonymous file share that leaked a hint, and an old, unpatched version of a system tool (`sudo`) that let a low-privilege user become root.

Methodology

The test followed the standard order any ethical hacker uses:

1. **Reconnaissance / Scanning** – find out what's running on the target.
2. **Enumeration** – poke at each open service to see what it reveals.
3. **Initial Access** – use whatever's exposed to get a foothold (a basic user account).
4. **Privilege Escalation** – turn that basic foothold into full admin/root access.
5. **Proof of Success** – capture "flags" (proof files) to confirm each stage was actually breached.

Tools Used

Tool	Purpose
Nmap	Scans a machine to find which network ports/services are open
FTP client	Connects to file-transfer servers, including anonymous ones
SSH client	Remote secure login to a machine
Bash/Linux shell	Used to explore the filesystem and run commands once logged in
A public sudo exploit script (originally written by "kingcope", 2010)	Takes advantage of a known bug in old sudo versions to gain root

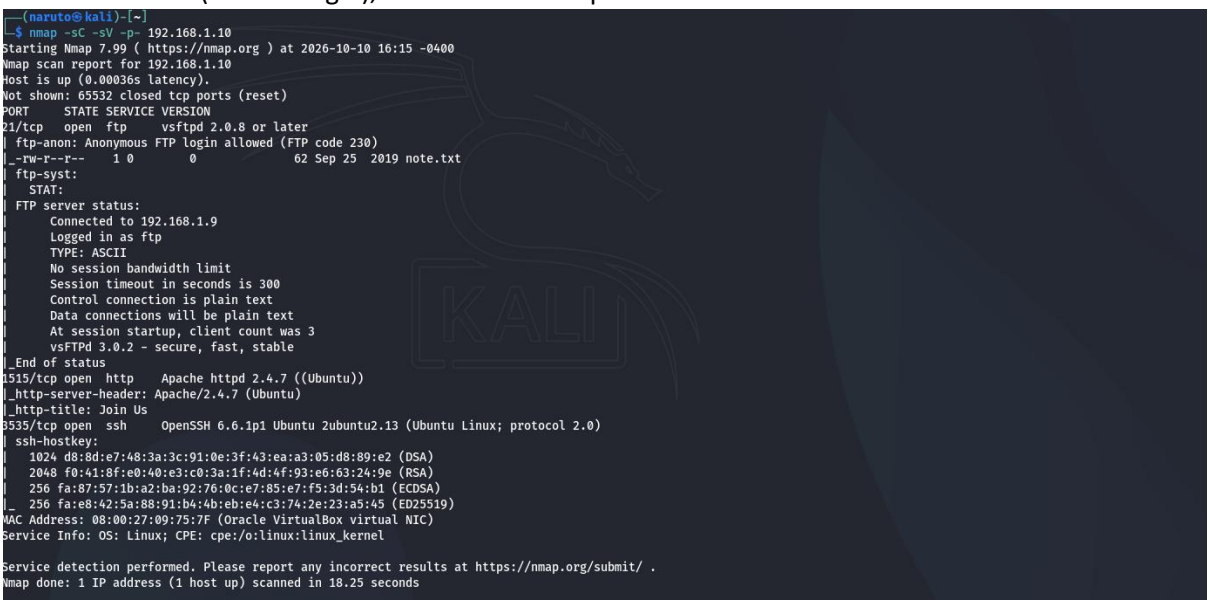
Step-by-Step Findings & Evidence

1. Port scan reveals three open doors

Running `nmap -sC -sV -p- 192.168.1.10` showed three open services:

- Port **21** – FTP (file server), running vsftpd, with **anonymous login allowed**
- Port **1515** – a website (Apache)

- Port **3535** – SSH (remote login), on a non-default port



2. Anonymous FTP leaks a clue

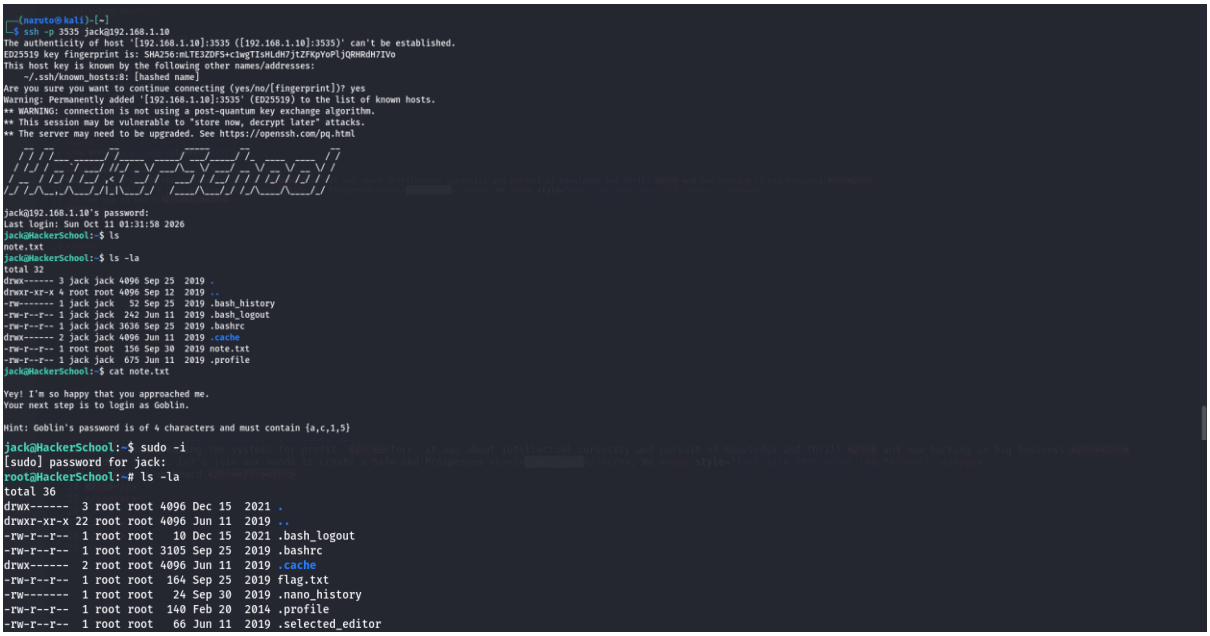
Since FTP allowed anonymous login, the tester connected with no real credentials and found a file `note.txt` in the root folder. It read: "Hello Dear! Planning to join our team? Try to login as Jack." This directly told an attacker which username to try next — a server should never leave hints like this reachable by anyone.



3. Logged in as "jack" over SSH(password found in the web interface bold letters (Cyberspace))

Using the leaked username, the tester connected via SSH on port 3535. The login succeeded, meaning Jack's password was weak or guessable. Inside Jack's home folder was another note: "Your next step is to login as Goblin. Hint: Goblin's password is 4 characters and must contain {a,c,1,5}." This note handed over both a username and a shortcut to the password — a serious information-disclosure problem.

ssh -p 3535 jack@192.168.1.10



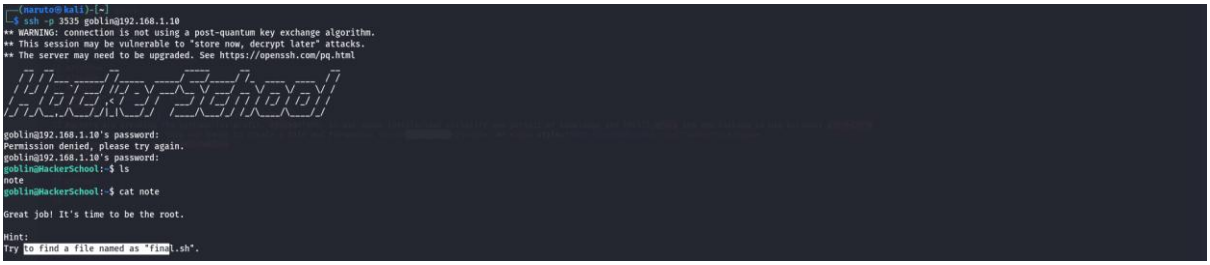
4. Jack had excessive privileges

The tester ran `sudo -i` while logged in as Jack — and it worked, dropping straight into a root shell with no restriction. This is a critical misconfiguration: Jack should never have had unrestricted admin rights. From there, `cat flag.txt` confirmed full compromise, printing the first proof-of-access flag (`root-flag{a9dcac68bfa4b4f6319e09fa041e3cb3}`).



5. Second path in — the "goblin" account

Following the password hint, the tester also logged in as `goblin` over SSH (first attempt failed, second succeeded — confirming the weak, guessable 4-character password). Goblin's home folder held a note: "Great job! It's time to be the root. Hint: Try to find a file named 'final.sh'."



6. A forgotten exploit script, left world-readable

Searching the filesystem (`find / -name "final.sh"`) turned up `/usr/share/hs/final.sh` — owned by root, but readable and executable by any user. Reading it revealed it was a public, well-known exploit for old sudo versions (affecting sudo 1.6.x before 1.6.9p21 and 1.7.x before 1.7.2p4 — a bug from 2010). Leaving a working root-exploit script on the box, accessible to unprivileged users, effectively handed out the keys to anyone who found it.

Attack Chain Summary

```
Anonymous FTP access
   -> leaked note pointing to user "jack"
      -> SSH login as jack (weak password)
         -> sudo misconfiguration -> direct root access (Flag #1)
      -> note pointed to user "goblin" with a weak-password hint
         -> SSH login as goblin
            -> found legacy sudo exploit script left on disk
               -> used it to escalate to root
```

Root Causes

- Anonymous FTP enabled on a server that shouldn't allow it.
- Secrets/hints stored in plaintext files (`note.txt`) reachable by unauthenticated or low-privilege users.
- Weak, guessable passwords for both `jack` and `goblin`.
- Excessive sudo privileges granted to a standard user (`jack`).
- Outdated software (`sudo`) vulnerable to a 15+ year old public exploit.
- A known exploit tool left on the server itself, with permissions that let any user read and run it.

Recommended Remediation

- Disable anonymous FTP access entirely, or restrict it to only files meant to be public.
- Remove all hint/credential files (like `note.txt`) from the server — never store passwords or login hints in accessible files.

- Enforce a strong password policy: minimum length, complexity, and no dictionary-guessable passwords.
- Review and restrict sudo privileges — apply least privilege; no standard user should have blanket `sudo -i` access.
- Patch and update sudo (and all system packages) to current versions to close known CVEs.
- Audit the filesystem for leftover tools/scripts (like `final.sh`) and delete anything not required for production; lock down file permissions so only authorized admins can read/execute sensitive scripts.
- Enable logging/monitoring for SSH logins and sudo usage so this kind of lateral movement would be caught in real time.

Note

This appears to be a deliberately built practice/training machine (the gamified flags and "note" hints are a strong signal), not a live production system — this reads as a lab walkthrough rather than a real-world incident report.